

Design Pattern — Singleton

Adam HAMOUC

May 2026

1. Intention

Garantir qu'une classe n'a **qu'une seule instance** durant toute la vie du programme, et fournir un point d'accès global à cette instance.

Équivalent fonctionnel d'une variable globale, mais encapsulé dans une classe avec contrôle d'accès.

2. Structure

- Constructeur **privé** : personne ne peut instancier la classe de l'extérieur.
- Méthode statique `getInstance()` : point d'accès unique, crée l'instance si elle n'existe pas encore.
- Instance stockée en **variable statique locale** (C++11) : initialisation lazy, thread-safe garanti par le standard.

3. Implémentation C++ moderne (Meyers Singleton)

```
class AudioEngine {
public:
    // Point d'accès unique
    static AudioEngine& getInstance() {
        static AudioEngine instance; // init au premier appel, thread-safe C++11
        return instance;
    }

    void playSound(const std::string& name) { /* ... */ }
    void setMasterVolume(float v) { m_volume = v; }

    // Supprimer copie et move : une seule instance possible
    AudioEngine(const AudioEngine&) = delete;
    AudioEngine& operator=(const AudioEngine&) = delete;

private:
    AudioEngine() = default; // constructeur privé
    float m_volume = 1.0f;
};

// Usage
AudioEngine::getInstance().playSound("explosion.wav");
AudioEngine::getInstance().setMasterVolume(0.8f);
```

Correction importante : la solution thread-safe n'est **pas** de "faire de l'instance un mutex". La solution correcte en C++11 est d'utiliser une **variable statique locale** dans `getInstance()` — le standard garantit que son initialisation est atomique et thread-safe, sans aucun mutex manuel.

Un mutex est utile si les *méthodes* de l'instance sont appelées depuis plusieurs threads simultanément (ex: `playSound()` depuis le thread audio et le thread jeu en même temps) — c'est un problème séparé du Singleton lui-même.

4. Cas d'usage en engine dev

Les rares cas où un Singleton se justifie : systèmes **vraiment uniques** par nature et dont l'accès global est inévitable.

- **AudioEngine** : un seul contexte audio par application.
- **Logger / Profiler** : collecte centralisée de données depuis tout le moteur.
- **InputManager** : un seul récepteur des events clavier/souris/gamepad.
- **ResourceManager** : cache global de textures, meshes, shaders.

En Unreal Engine, le Singleton est implicitement utilisé via **GEngine**, **GWorld**, ou les **Subsystems** (la version propre recommandée par Epic).

5. Pourquoi c'est un anti-patron

- **Dépendance cachée** : n'importe quel fichier peut appeler `getInstance()` sans que la dépendance soit visible dans les interfaces — rend le code difficile à lire et à tracer.
- **Réduit la modularité** : impossible de remplacer l'instance par un mock pour les tests unitaires.
- **Ordre de destruction imprévisible** : si deux singletons dépendent l'un de l'autre, l'ordre de destruction à la fin du programme est indéfini.
- **Encourage le couplage fort** : les systèmes communiquent directement via l'instance globale au lieu de passer par des interfaces.

Alternative recommandée : injection de dépendances — passer le système en paramètre à ceux qui en ont besoin. Plus verbeux, bien plus testable et modulaire.

6. Quand utiliser / éviter

Utiliser (avec parcimonie) :

- Système réellement unique par contrainte matérielle/OS (audio, input).
- Outil transversal sans état partagé critique (logger, profiler).
- Prototype rapide où la modularité n'est pas encore une priorité.

Éviter :

- Dès que le système doit être testable ou remplaçable.
- Si plusieurs instances pourraient être utiles à terme (ex: deux fenêtres de rendu).
- Dans du code engine de production — préférer les Subsystems (UE) ou l'injection.